# Danila Nelin

## Junior DevOps / SRE Engineer

t.me/spaster nelindanila7@gmail.com github.com/spasster

Moscow, hybrid or remote · MTUCI, 09.03.02 “DevSecOps”, 1st year · English B2

## ABOUT

I ship services to production myself and keep them running: deploys, alerts, dashboards, backups. My production runs on Docker Compose: SolMeme has been live for about a year, SolPump is in beta with full monitoring. I was the tech lead of a 4-person team that built a study observability and self-healing setup on Kubernetes. Looking for an internship or a junior DevOps / SRE role.

## SKILLS

| | |
|---|---|
| **CI/CD** | GitHub Actions: lint, tests, Docker image builds, SSH deploy only from main and only when everything is green (SolPump) |
| **Servers and backups** | Linux, Docker Compose, Nginx, systemd; Ansible roles for Linux system log collection (rsyslog, auditd) in MTC SRE; daily mongodump, 7 copies kept |
| **Monitoring and alerting** | Prometheus, Alertmanager, Grafana, Loki, Tempo, Sentry. SolPump: 14 alert rules, 6 dashboards. MTC SRE: 12 rules, 5 dashboards, metric → trace → log correlation |
| **Kubernetes** | Study setup on k3s: Helm, Istio with strict mTLS, Chaos Mesh, self-healing with hard limits. Production runs on Docker Compose |
| **SLO** | MTC SRE: at least 99.5% successful requests, p95 under 300 ms, external Blackbox probes; separate error-budget burn rules |
| **Load and chaos** | k6: Lotty — 50 users, target under 1% errors and p95 under 2 s; MTC SRE — capacity steps and load spikes, chaos scenarios (pod kill, slow pod, network partition, bad release) |
| **Documentation** | 7 runbooks in MTC SRE; C4 diagrams and a runbook in Lotty |

## STACK

| | |
|---|---|
| **Infrastructure** | Linux, Docker Compose, Nginx, systemd, Ansible, GitHub Actions |
| **Observability** | Prometheus, Alertmanager, Grafana, Loki, Alloy, Tempo, Blackbox, Sentry |
| **Kubernetes** | k3s (k3d), Helm, Istio, Chaos Mesh — study setup |
| **Languages** | TypeScript, Python — confident; Go — basic |
| **Data and tests** | PostgreSQL, MongoDB, Redis, ClickHouse; pytest, Jest, k6 |

## PROJECTS

### SolMeme and SolPump — Solana token launchpad and price prediction (from scratch, solo, ~1 year in production)

sol-meme.com · solpump.gg (beta)

Anyone can launch their own token in a couple of minutes: upload an image, confirm in the wallet and get a token, a trading pool and a referral link. SolPump grew out of it: short-term bets on whether a meme token’s price goes up or down.

- One-year result: users created 619 tokens and 459 pools; the service earned 321 SOL in token-burn fees.
- Role. Solely responsible for running the services: server, Nginx, Docker, deploys, monitoring, backups.
- Automated deploys: lint, tests, image builds, then SSH to the server — only if every previous step passed. Daily database backup, 7 copies kept.
- For SolPump I wrote 14 alert rules and built 6 dashboards: business metrics, HTTP, backend, host, containers, Prometheus itself. Job queues export metrics and have alerts too.
- Errors go to Sentry, logs to Loki. Alerts fire if any withdrawal fails, the price stops updating or the fee balance goes negative.

### MTC SRE — study observability and self-healing setup on Kubernetes

github.com/spasster/MTC-SRE · “Telecom of the Future 2026” hackathon, MTS case #2, 2nd place · team of 4, study stand, not production · tech lead

A demo shop: the checkout request stores the cart in Redis and calls a payment service. The platform detects slow requests and 5xx errors, correlates metric, trace and log, and brings the service back to normal without taking dangerous actions blindly.

- Role (tech lead of 4). Architecture, SLOs, self-healing thresholds and limits, technical review. Services and dashboards, chaos and load tests, the cluster, CI and access rights were built by teammates.
- SLO: at least 99.5% successful requests (5xx at most 0.5% per minute), p95 under 300 ms, Redis and payment alive per Blackbox probes. Rules evaluated every 15 s.
- Self-healing. Automatic, after a dry run: remove a pod at least 3× slower than the rest; scale checkout from 2 to 5 replicas when all are CPU-bound. Release rollback and Redis restart — only via a Telegram button. At most 2 actions per incident.
- In numbers: 12 Prometheus rules, 5 Grafana dashboards, 7 runbooks, 276 pytest tests, 6 chaos scenarios; full e2e on clean Ubuntu.

### Lotty — A/B testing platform (PROD final stage)

github.com/spasster/lotty-ab-platform · backend and infrastructure, solo

An internal service for teams: roll features out to a share of users, compare variants, view reports and automatically stop an experiment when a key metric drops.

- Role. All infrastructure and observability: Docker Compose, 15 Prometheus metrics, 5 Grafana dashboards, structured logs.
- CI: lint, tests with coverage, build, e2e tests against real Mongo, Redis and ClickHouse, image builds, smoke check after compose up.
- k6 load test (50 users, 30 seconds) and a quick post-deploy test: health, ready, login, decide, events.

## CONTESTS AND HACKATHONS

- PROD 2026 — finalist. Individual stage: 65/100. Final-stage case: an A/B service that stops an experiment when a metric drops (my project, Lotty, above).
- “Telecom of the Future 2026” (MTUCI), MTS case #2 “SRE observability platform” — 2nd place. MTC SRE project above: team of 4, I was tech lead.
- Technostrelka 2025 — 1st place. Neymark case: a platform for student dormitories.
- Hackathons in Ryazan — 2nd and 3rd place.
- IT-Quantum. Telegram bots, computer vision (face mask detection), an Android app for transit payments.